Application Servers - Context

SLIDE 1

Welcome! Today we're meeting three Java application servers. They all run your code, but each one has a very different personality — think of them as three characters in the same story.

SLIDE 2

A static page can say Hello all day — but real applications need to log users in, talk to a database, and handle money. That job is too big for a simple web server. Today we compare the three servers that do this job in the Java world.

SLIDE 3

Here's the definition, once: an application server hosts your Java code and takes care of security, transactions, and database connections. If the website is the dining room, the application server is the kitchen — where the real cooking happens.

SLIDE 4

Splitting it simply: the web server is the waiter — it carries plates of HTML and images to the browser. The app server is the chef — it actually cooks, running code and talking to databases. [pause for the joke] The waiter never burns the food... because he never cooks.

SLIDE 5

Section break. From here, one slide at a time: our three contenders — Tomcat, GlassFish, and WildFly. Same language, three attitudes.

SLIDE 6

First up, Tomcat. It's a lightweight servlet container: Servlets and JSP, nothing heavier — no EJB, no JMS, no JPA built in. That's not a weakness, it's a choice: less to configure, starts fast. It's the scooter of Java servers — light, nimble, parks anywhere.

SLIDE 7

So when do you pick it? Strengths: light, simple, easy to configure.

Typical use: simple apps and dynamic websites. If your project is 'a form, a database, and a deadline', Tomcat is your friend — start small.

SLIDE 8

Now the opposite personality: GlassFish. It's the complete Java EE application server — Servlets, JSP, EJB, JPA, JMS, web services, all of it. Where Tomcat says 'keep it light', GlassFish says 'everything included'. It's the Swiss Army knife that comes with a kitchen.

SLIDE 9

Three quick facts. Administration: a web console or config files, your choice. Strength: fully standards-compliant with Java EE — good for exams and enterprise contracts. Typical use: enterprise apps that need EJB and the full services stack. Standards first.

SLIDE 10

Third contender: WildFly — the modern, Jakarta EE server. Full feature support, plus clustering and high availability. And here's the fun part: it never sleeps — if one node goes down, another takes over. Always on.

SLIDE 11

Let's pin down WildFly's personality. Strengths: fast, flexible, and fully distributed — built from the ground up to run on a cluster of machines. Typical use: big, busy systems that can't afford to go down — banks, streaming, anything with real traffic. [pause for the joke] It never sleeps... so you finally can.

SLIDE 12

Time to choose. Tomcat is the scooter — light, quick, easy to park: small apps and sites. GlassFish is the Swiss Army knife — every tool in the box: full Java EE. WildFly never sleeps — for big, always-on loads. [ask the class] Which personality fits YOUR project? Remember: there is no best server — only the best fit for the job.

SLIDE 13

Recap in three lines: small job → Tomcat; need every Java EE feature → GlassFish; huge, always-on cluster → WildFly. Golden rule: match the server

to the job — don't use a truck to deliver a pizza. [open the floor] Your turn — which one fits your project? Questions?

BIG TOPICS - DEFINITIONS

BIG TOPICS - DEFINITIONS

1. Application Server

Definition: Software that hosts and runs application code (business logic) and provides services like security, transaction management, and database connection pooling.

2. Web Server vs Application Server

Definition: Web Server delivers static content (HTML, images, CSS). Application Server executes dynamic code and handles business logic.

3. Servlet Container

Definition: A lightweight runtime that manages Java Servlets and JSP pages — receives HTTP requests, runs the Java code, and returns responses.

4. Apache Tomcat

Definition: Lightweight, open-source Java servlet container that supports Servlets and JSP.

5. GlassFish

Definition: Complete, full-profile Java EE application server supporting Servlets, JSP, EJB, JPA, JMS, and Web Services.

6. WildFly (formerly JBoss AS)

Definition: Modern, fast, lightweight full Jakarta EE / Java EE application server with built-in clustering and high availability.

7. Java EE / Jakarta EE (now Jakarta EE)

Definition: Set of standard specifications for enterprise Java — includes Servlets, JSP, EJB, JPA, JMS, Web Services, security, transactions.

8. Key Java EE Technologies Mentioned:

- Servlet: Java class that handles HTTP requests and generates dynamic responses.
- JSP (JavaServer Pages): HTML pages with embedded Java for dynamic content; runs

on all three servers.
- EJB (Enterprise JavaBeans): Component for business logic, transactions, security — supported by GlassFish and WildFly, NOT by Tomcat.
- JPA (Java Persistence API): Standard for mapping Java objects to database tables — supported by GlassFish and WildFly, NOT by Tomcat out of the box.
- JMS (Java Message Service): Messaging system for async communication between apps — supported by GlassFish and WildFly, NOT by Tomcat.
- Web Services: Standard way for apps to talk to each other over HTTP/SOAP/REST — included in GlassFish full stack.

9. Clustering and High Availability

Definition: Running the same app on multiple servers (nodes) together so if one fails, another takes over with no downtime.

10. How to Choose - Best Fit Rule

Definition: No best server — only best fit for the job (Slide 12-13).